

W1 PRACTICE

PART 2 – MANIPULATION

⚠ Before this practice

- You need to have completed: **PART 1- EXPLORATION**

🧠 What do you need to **be able to do** for next week?

- Identify the **type** of variable
- Manipulate **nullable** and **non-nullable** variables.
- Differentiate between **final** and **const**.
- Manipulate **strings, lists, and maps**.
- Use **loops** and **conditions** to control flow.
- Define and call functions with **positional** and **named arguments**, understand **arrow syntax**



L0 NO AI

Solve problems entirely on your own.

📖 Are you lost?

Read the following documentation to be ready for this practice:

- | | | |
|----------------------------------|------------------------------|-----------------------------|
| ✓ Variables | ✓ Lists | ✓ Functions |
| ✓ Null Safety | ✓ Loops | |
| ✓ Built-in types | ✓ Conditions | |



EX 1 – Manipulate Types

Are you clear about strings, integer, list, map, set, objects in Dart?

Examine the given data structures and **write the inferred Dart type** for each one (see example)

Notes

- First find by yourself the type
- If you need, double check your answer with VSCode.

Data	Dart Type
<pre>const studentGrades = { 'Sokan': [90, 85, 88], 'Sokea': [70, 80, 75], 'Hay': [95, 92, 89], };</pre>	Map<String, List<int>>
<pre>const pizzaPrices = { 'margherita': 5.5, 'pepperoni': 7.5, 'vegetarian': 6.5, };</pre>	Map<String, double>
<pre>const books = [{'title': '1984', 'author': 'George Orwell'}, {'title': 'Brave New World', 'author': 'Aldous Huxley'}, {'title': 'Fahrenheit 451', 'author': 'Ray Bradbury'},];</pre>	List<String, String>
<pre>const company = { 'HR': {'employees': 5, 'budget': 20000}, 'IT': {'employees': 10, 'budget': 50000}, 'Marketing': {'employees': 7, 'budget': 30000}, };</pre>	Map<String, Map<String, int>>
<pre>const coordinates = [[1, 2], [3, 4], [5, 6],];</pre>	List<List <int>>
<pre>const productDetails = { 'id': 101, 'name': 'Laptop', 'price': 999.99, 'inStock': true, };</pre>	Map<String, object>
<pre>const operations = [(int a, int b) => a + b, (int a, int b) => a - b, (int a, int b) => a * b,];</pre>	List<int function(int, int)>
<pre>const distances = {3.1, 5.5, 10.2, 7.8};</pre>	Set double
<pre>const university = { 'departments': [{ 'name': 'Computer Science', 'students': [{'name': 'Alice', 'age': 22}, {'name': 'Bob', 'age': 24},] }, { 'name': 'Mathematics', 'students': [{'name': 'Charlie', 'age': 21}, {'name': 'Dave', 'age': 23},] }] };</pre>	Map<String, List<Map<String, Object>>

EX 2 – Manipulate *final* and *const*

In this exercise, you need to decide which variable can be declared as **const** or **final**.

```
// 1 - startText
String iLike = 'I like pizza';

// 2 - toppings
String toppings = 'with tomatoes';
toppings += " and cheese";

// 3 - message
String message = '$iLike $toppings';
print(message);

// 4 - rgbColors
List<String> rgbColors = ['red', 'blue', 'green'];
print(rgbColors);

// 5 - weekDays
List<String> weekDays = ['monday', 'tuesday', 'wednesday'];
weekDays.add('thursday');
print(weekDays);

// 6 - scores
List<int> scores = [45,35,50];
scores = [45,35,50, 78];
print(scores);
```

Guess which variables can be declared as **const**, **final** or **var**, and explain your choices.

Notes

- Read [here](#) to understand the concepts.
- Prefer *const* over *final* over *var*.

	VAR, FINAL, CONST?	WHY
iLike	CONST	<i>Because this variable never changes</i>
toppings	FINAL	<i>Because this variable need to add-on</i>
message	FINAL	<i>When we adjust the toppings message need to change as well.</i>
rgbColors	CONST	<i>Its is a static define that couldn't change</i>
weekDays	FINAL	<i>We need to add the remain week day.</i>
score	VAR	<i>For score – we can store it in many type int, double and float</i>

EX 3 – Filter a list

Write a Dart program that takes a list of exam scores and **returns the scores of all students who passed**

Given

- You are given a list of integers representing students' exam scores.
- A student **passes** if their score is **50 or higher**.

Example input:

[45, 78, 62, 49, 85, 33, 90, 50]

Example output:

[78, 62, 85, 90, 50]

Requirements

Implement the filtering in **two different ways**.

Method 1 — Using a for loop

- Use a for loop to examine each score.

Add passing scores (≥ 50) to a new list.

```
void method1(List<int> score) {  
    List<int> passScore = [];  
    int size = score.length;  
    for (var i = 0; i < size; i++) {  
        if (score[i] >= 50) {  
            passScore.add(score[i]);  
        }  
    }  
    print("method 1:");  
    for (var i = 0; i < passScore.length; i++) {  
        print(passScore[i]);  
    }  
}
```

•

Method 2 — Using where()

- Use **where** function to filter the list.
- Pass a **proper anonymous function** to where().

```
void method2(List<int> score) {  
    List<int> passScore = score.where((e) => (e >= 50)).toList();  
  
    print("method 2:");  
    for (var i = 0; i < passScore.length; i++) {  
        print(passScore[i]);  
    }  
}
```